Table des matières

Labo 10 — Passer à un format de table ouvert 1
Avant de commencer — les bibliothèques des tables ouvertes 2
Étape 1 — Ce que le bronze ne sait pas faire 3
Étape 2 — Le catalogue 4
Étape 3 — Charger la télémétrie jusqu'à la fin de 2025 6
Étape 4 — Lire la table avec DuckDB 10
Étape 5 — Les trois mois de 2026, jour par jour 12
Étape 6 — Remonter au 31 décembre 2025 15
Étape 7 — Compacter, expirer, nettoyer 18
Étape 8 — Les séries de l'usine en argent 21
Étape 9 — La même télémétrie en Delta Lake 24
Étape 10 — Versionner 29
Récapitulatif 29
Dépannage 30
Annexe — Sur Snowflake 31
Annexe — Sur Databricks 33

Labo 10 — Passer à un format de table ouvert

Labo · Cours 10/12 — Formation Data Engineering

Ce que vous allez faire. Faire de la télémétrie du bronze une table Iceberg : un catalogue, des instantanés, un schéma qui évolue au passage du micrologiciel 2.0 sans réécrire un seul fichier. La lire par DuckDB, remonter le temps jusqu'au 31 décembre 2025, puis payer ce que la table coûte en entretien : compacter, expirer, nettoyer. Mettre ensuite les séries de l'usine en Iceberg, et charger la même télémétrie en Delta Lake pour comparer les deux formats. Objectif de fin : l'état au 2025-12-31 relu après l'entretien, 158 146 messages, identiques au bronze de cette date.

Le cours tient en une séance, sans Docker.

Deux terminaux servent. L'Anaconda Prompt, environnement `formations` actif, lance Python et Git, depuis `Data_Domain\Projet_Boreal_intermediaire\01_de` : c'est là que vivent le code et sa configuration. La ligne de commande DuckDB, lancée depuis

© 2026 iAlexMG. Tous droits réservés.

`Data_Domain\Projet_Boreal_intermediaire`, lit les tables et les compare au bronze : les chemins de ses requêtes partent de là. Ubuntu ne sert pas à ce cours, parce qu'aucun service ne tourne : une table ouverte, ce sont des fichiers, et les programmes qui les écrivent ou les lisent n'ont besoin de rien d'autre.

La liste de contrôle :

- ☐ PyIceberg 0.12 et deltalake 1.6 installés ; les extensions `iceberg`, `delta` et `sqlite` de DuckDB installées
- ☐ Le bronze de la télémétrie relu : 933 fichiers, 204 485 messages, deux noms pour la même température
- ☐ Le catalogue créé, avec ses deux espaces de noms, `bronze` et `argent`
- ☐ La télémétrie chargée jusqu'à la fin de 2025 : 28 instantanés, 158 146 messages, une colonne renommée et une ajoutée au 2025-05-01
- ☐ La table lue par DuckDB : une seule colonne de température, et un seul fichier ouvert sur 28 pour octobre 2025
- ☐ Les trois mois de 2026, jour par jour : 90 instantanés de plus, 204 485 messages, et l'instantané de rang 28 étiqueté `fin-2025`
- ☐ Le voyage dans le temps : 158 146 messages à l'étiquette, et 0 écart dans les deux sens avec le bronze arrêté au 2025-12-31
- ☐ L'entretien : 90 fichiers compactés en 3, 122 instantanés expirés, 90 fichiers effacés, et l'étiquette qui se relit encore
- ☐ L'argent en Iceberg : 240 240 lignes dans `usine_horaire`, 0 écart avec l'entrepôt
- ☐ La même télémétrie en Delta : 31 versions, deux colonnes de température, le renommage refusé, et la version 27 illisible après le nettoyage

---

Avant de commencer — les bibliothèques des tables ouvertes

Ce cours écrit ses tables par deux bibliothèques Python que le reste de la formation n'emploie pas : PyIceberg, avec son catalogue SQL et le module compilé qui calcule les partitions, et `deltalake`. Elles sont dans le `requirements.txt` de la formation. Vérifiez qu'elles sont là, dans l'Anaconda Prompt, depuis `Projet_Boreal_intermediaire`, et installez-les sinon :

```
python -c "import pyiceberg, pyiceberg_core, deltalake; print(pyiceberg.__version__, deltalake.__version__)"
pip install -r ..\01_Data_Engineering\requirements.txt
```

La première commande doit afficher `0.12.0 1.6.5`, ou des versions plus récentes de la même série.

DuckDB lit ces tables par trois extensions : `iceberg`, `delta`, et `sqlite`, qui lui fera lire le catalogue. Ouvrez la ligne de commande DuckDB depuis `Projet_Boreal_intermediaire`, et installez-les une fois pour toutes :

```
INSTALL iceberg;
INSTALL delta;
```


© 2026 iAlexMG. Tous droits réservés.

```
INSTALL sqlite;
SELECT extension_name, installed, extension_version
FROM duckdb_extensions()
WHERE extension_name IN ('iceberg', 'delta', 'sqlite_scanner')
ORDER BY ALL;
```

extension_name varchar	installed boolean	extension_version varchar
delta	true	45c4087
iceberg	true	45163a28
sqlite_scanner	true	f79b1db

L'extension `sqlite` s'appelle `sqlite_scanner` dans la liste, et les numéros de version sont ceux de DuckDB 1.5.5 : les vôtres suivront votre DuckDB. Gardez cette ligne de commande ouverte pendant toute la séance.

Étape 1 — Ce que le bronze ne sait pas faire

Théorie : `Theorie_10_DE.md`, section 1.

Le bronze de la télémétrie, écrit au Cours 02, est un dossier de Parquet : un fichier par jour de réception, et rien d'autre. Dans la ligne de commande DuckDB, comptez-les, puis regardez comment la température du moteur y est rangée, micrologiciel par micrologiciel :

```
SELECT COUNT(*) AS fichiers FROM glob('plateforme/lac/bronze/telemetrie/**/*.parquet');

SELECT message.micrologiciel AS micrologiciel, COUNT(*) AS messages,
       COUNT(message.temp_moteur_max) AS temp_moteur_max,
       COUNT(message.temperature_moteur_max) AS temperature_moteur_max,
       COUNT(message.heures_moteur) AS heures_moteur,
       MIN(jour) AS premier_jour, MAX(jour) AS dernier_jour
FROM read_parquet('plateforme/lac/bronze/telemetrie/**/*.parquet', hive_partitioning = true)
GROUP BY ALL ORDER BY ALL;
```

fichiers int64
933

© 2026 iAlexMG. Tous droits réservés.

micrologiciel varchar	messages int64	temp_moteur_max int64	temperature_moteur_max int64	heures_moteur int64	premier_jour date	dernier_jour date
1.3.0	71029	71029	0	0	2023-09-09	2025-07-14
1.6.2	17649	17649	0	0	2024-09-05	2025-07-13
2.0.1	115807	0	115807	115807	2025-05-01	2026-03-31

Deux choses se lisent là. Le micrologiciel 2.0 a renommé la température du moteur et ajouté les heures du moteur ; le bronze, qui déclare les deux noms depuis le Cours 02, les garde côte à côte, chacun vide pour l'autre moitié des messages. Tout lecteur doit s'en souvenir et écrire `COALESCE(temperature_moteur_max, temp_moteur_max)`, pour toujours. Et la bascule n'a pas été franche : les véhicules restés en 1.3 ou en 1.6 ont encore publié sous l'ancien nom jusqu'à la mi-juillet 2025.

Ce que le dossier ne dit pas compte autant. Rien n'y indique ce que contenait le bronze le 31 décembre au soir, si ce n'est le nom des dossiers ; un fichier réécrit remplace l'ancien sans laisser de trace ; et un lecteur qui liste le dossier pendant qu'on y écrit voit ce qui est là à cet instant, fichier à moitié écrit compris. C'est ce qu'une table ouverte vient régler.

Étape 2 — Le catalogue

Théorie : `Theorie_10_DE.md`, section 3.

Les tables vivent dans un paquet neuf du dépôt, `tables`, à côté de `pipeline` et de `flux`. Il réemploie la configuration du pipeline : ouvrez `config.toml`, dans `Projet_Boreal_intermediaire\01_de`, et ajoutez deux lignes à la section `[chemins]`, sous `flux` :

```
# Les tables ouvertes du Cours 10 : l'entrepôt Iceberg et son catalogue, puis les tables Delta.
iceberg = "plateforme/lac/iceberg"
delta = "plateforme/lac/delta"
```

Créez le paquet, depuis l'Anaconda Prompt, dans `01_de` :

```
mkdir tables
notepad tables\__init__.py
```

Ce code ne se tape pas dans la ligne de commande DuckDB : ce sont des fichiers du dépôt. Au moment d'enregistrer, choisissez « Tous les fichiers » dans la liste des types, sans quoi le Bloc-notes ajoute `.txt` au nom. Le premier fichier ne tient qu'une ligne :

```
# -*- coding: utf-8 -*-
"""tables : les formats de table ouverts du lac de Boréal, Iceberg puis Delta."""
```

Le deuxième, `notepad tables\catalogue.py`, ouvre le catalogue :

© 2026 iAlexMG. Tous droits réservés.

```python
# -*- coding: utf-8 -*-
"""catalogue.py : le catalogue Iceberg du lac, une base SQLite qui tient un pointeur par table."""
from pyiceberg.catalog.sql import SqlCatalog

from pipeline.config import chemin


def catalogue() -> SqlCatalog:
    """Le catalogue du lac, avec ses deux espaces de noms, bronze et argent."""
    racine = chemin("iceberg")
    racine.mkdir(parents=True, exist_ok=True)
    cat = SqlCatalog("boreal", uri=f"sqlite:///{(racine / 'catalogue.db').as_posix()}",
                     warehouse=racine.as_posix())
    for espace in ("bronze", "argent"):
        cat.create_namespace_if_not_exists(espace)
    return cat
```

Un catalogue SQL est une base de données ordinaire, ici un fichier SQLite, `catalogue.db`, rangé à la racine de l'entrepôt Iceberg. Le nom `boreal` le désigne dans PyIceberg ; `warehouse` dit où écrire les tables. Le chemin passe par `as_posix()`, avec des barres obliques : c'est la forme que PyIceberg accepte sous Windows (voir le dépannage). Les deux espaces de noms (*namespaces*), les équivalents des schémas d'une base, se créent au premier appel et jamais deux fois.

Appelez-le une fois, depuis `01_de` :

```
python -c "from tables.catalogue import catalogue; c = catalogue(); print(sorted(c.list_namespaces()), c.list_tables('bronze'))"
```

```
[('argent',), ('bronze',)] []
```

Deux espaces, aucune table. Dans la ligne de commande DuckDB, ouvrez le catalogue comme n'importe quelle base SQLite, en lecture seule :

```
ATTACH 'plateforme/lac/iceberg/catalogue.db' AS catalogue (TYPE sqlite, READ_ONLY);
SELECT name, column_names FROM (SHOW ALL TABLES) WHERE database = 'catalogue';
```

```
┌──────────────────────────────┬──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│             name             │                                                 column_names                                                 │
│           varchar            │                                                  varchar[]                                                   │
├──────────────────────────────┼──────────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│ iceberg_namespace_properties │ [catalog_name, namespace, property_key, property_value]                                                      │
│ iceberg_tables               │ [catalog_name, table_namespace, table_name, metadata_location, previous_metadata_location, iceberg_type] │
```


© 2026 iAlexMG. Tous droits réservés.

```
└──────────────────────────────┴──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

Tout le catalogue tient dans ces deux tables. La seconde aura une ligne par table Iceberg, et sa colonne qui compte est `metadata_location` : le chemin du fichier qui décrit la table maintenant. Écrire dans une table, c'est finir par changer ce chemin.

Étape 3 — Charger la télémétrie jusqu'à la fin de 2025

Théorie : `Theorie_10_DE.md`, sections 2 et 4.

Le chargeur lit le bronze par DuckDB, aplatit le message en colonnes, et l'ajoute à la table une période à la fois : un mois par ajout, ou un jour. Créez-le, `notepad tables\telemetrie_iceberg.py`. D'abord ce qu'il importe, et les deux dates qui comptent :

```
# -*- coding: utf-8 -*-
"""telemetrie_iceberg.py : la télémétrie du bronze en table Iceberg, un instantané par période.

Usage, depuis 01_de : python -m tables.telemetrie_iceberg --jusqua AAAA-MM [--par-jour]
"""
import argparse
import time
from datetime import date, timedelta

import duckdb
from pyiceberg.exceptions import NoSuchTableError
from pyiceberg.transforms import MonthTransform
from pyiceberg.types import DoubleType

from pipeline.config import chemin
from tables.catalogue import catalogue

TABLE = "bronze.telemetrie"
MICROLOGICIEL_2 = date(2025, 5, 1)          # la diffusion du micrologiciel 2.0
DEBUT = date(2023, 8, 1)                     # le premier jour livré, vide jusqu'au 2023-09-08
```

Puis la lecture d'une période, et le découpage en périodes :

```
def lire(con, debut, fin, deux):
    """Les messages reçus de debut (compris) à fin (exclu), à plat, sous le schéma de la table."""
    nom = "temperature_moteur_max" if deux else "temp_moteur_max"
    heures = "m.heures_moteur AS heures_moteur," if deux else ""
    return con.sql(f"""
        SELECT m.message_id AS message_id, m.sernr AS sernr, m.micrologiciel AS micrologiciel,
```


© 2026 iAlexMG. Tous droits réservés.

```
            m.debut AS debut, m.fin AS fin, m.distance_km AS distance_km, {heures}
            m.regime_moyen AS regime_moyen, m.regime_max AS regime_max,
            COALESCE(m.temperature_moteur_max, m.temp_moteur_max) AS {nom},
            m.temp_exterieure AS temp_exterieure, m.tension_batterie_min AS tension_batterie_min,
            m.position.lat AS lat, m.position.lon AS lon, m.codes AS codes,
            sujet, recu_le, qos, jour
        FROM (SELECT message AS m, * FROM bronze)
        WHERE jour >= DATE '{debut}' AND jour < DATE '{fin}'
        ORDER BY recu_le""").to_arrow_table()


def periodes(depuis, jusqua, par_jour):
    """Les tranches à charger : des mois, ou des jours."""
    d = depuis
    while d <= jusqua:
        mois_suivant = date(d.year + (d.month == 12), d.month % 12 + 1, 1)
        fin = d + timedelta(days=1) if par_jour else mois_suivant
        yield d, min(fin, jusqua + timedelta(days=1))
        d = fin


def nommer(debut, fin):
    """La période d'un ajout, pour l'historique : un jour, ou du premier jour au dernier."""
    dernier = fin - timedelta(days=1)
    return f"{debut}" if dernier == debut else f"{debut} au {dernier}"
```

`lire()` rend une table Arrow, le format en mémoire que DuckDB et PyIceberg partagent sans copie. Avant le 2025-05-01, la température garde son ancien nom et les heures du moteur n'existent pas ; à partir de là, elle prend le nouveau. Dans les deux cas, `COALESCE` prend la valeur où elle se trouve, puisqu'un véhicule resté en 1.x publie encore sous l'ancien nom en mai 2025.

Enfin la porte d'entrée, qui reprend là où la table s'est arrêtée :

```
def principal():
    p = argparse.ArgumentParser(description="La télémétrie du bronze en table Iceberg.")
    p.add_argument("--jusqua", required=True, help="dernier mois chargé, AAAA-MM")
    p.add_argument("--par-jour", action="store_true",
                   help="un instantané par jour plutôt que par mois")
    a = p.parse_args()
    annee, mois = map(int, a.jusqua.split("-"))
    jusqua = date(annee + (mois == 12), mois % 12 + 1, 1) - timedelta(days=1)

    con = duckdb.connect()
```


© 2026 iAlexMG. Tous droits réservés.

```
bronze = (chemin("lac") / "bronze" / "telemetrie").as_posix()
con.sql(f"CREATE VIEW bronze AS SELECT * FROM read_parquet('{bronze}/**/*.parquet', "
        "hive_partitioning = true)")
cat = catalogue()
try:
    table = cat.load_table(TABLE)
    jours = table.scan(selected_fields=("jour",)).to_arrow()["jour"].to_pylist()
    depuis = max(jours) + timedelta(days=1) if jours else DEBUT
except NoSuchTableError:
    table, depuis = None, DEBUT
```

Et la boucle des ajouts :

```
t0, ajouts, lignes = time.time(), 0, 0
for debut, fin in periodes(depuis, jusqua, a.par_jour):
    deux = debut >= MICROLOGICIEL_2
    donnees = lire(con, debut, fin, deux)
    if donnees.num_rows == 0:
        continue                          # un jour sans message ne fait pas d'instantané
    if table is None:
        table = cat.create_table(TABLE, schema=donnees.schema)
        with table.update_spec() as spec:
            spec.add_field("jour", MonthTransform(), "mois")
    noms = [c.name for c in table.schema().fields]
    if deux and "temp_moteur_max" in noms:
        with table.update_schema() as schema:
            schema.rename_column("temp_moteur_max", "temperature_moteur_max")
            schema.add_column("heures_moteur", DoubleType())
        print(f"{debut} : micrologiciel 2.0, colonne renommée et colonne ajoutée")
        noms = [c.name for c in table.schema().fields]
    table.append(donnees.select(noms),
                 snapshot_properties={"boreal.periode": nommer(debut, fin)})
    ajouts += 1
    lignes += donnees.num_rows
print(f"{ajouts} ajouts, {lignes} messages, en {time.time() - t0:.1f} s.")
if table is not None:
    table = cat.load_table(TABLE)
    print(f"La table : {len(table.snapshots())} instantanés, "
          f"{table.inspect.files().num_rows} fichiers de données.")
    print(f"Métadonnées courantes : {table.metadata_location}")
```


© 2026 iAlexMG. Tous droits réservés.

```
if __name__ == "__main__":
    principal()
```

Trois gestes y portent le cours. La table naît partitionnée par le mois de `jour`, par `MonthTransform` : aucune colonne `mois` n'existe, c'est la table qui dérive la partition. Au premier mois du micrologiciel 2.0, `update_schema()` renomme une colonne et en ajoute une, en une seule validation. Et chaque `append()` porte dans son résumé la période qu'il charge, `boreal.periode`, pour que l'historique se lise.

Lancez-le jusqu'à décembre 2025, un instantané par mois :

```
python -m tables.telemetrie_iceberg --jusqua 2025-12
```

```
2025-05-01 : micrologiciel 2.0, colonne renommée et colonne ajoutée
28 ajouts, 158146 messages, en 14.1 s.
La table : 28 instantanés, 28 fichiers de données.
Métadonnées courantes : C:/…/Projet_Boreal_intermediaire/plateforme/lac/iceberg/bronze/telemetrie/metadata/00030-….metadata.json
```

Le premier mois, août 2023, n'a aucun message, et n'a donc pas d'instantané : 28 mois vont de septembre 2023 à décembre 2025. Le chemin des métadonnées est absolu, et son suffixe, un identifiant tiré au hasard, sera autre chez vous. La durée aussi, d'un poste à l'autre.

Regardez ce que la table a écrit, dans la ligne de commande DuckDB :

```
SELECT CASE WHEN filename LIKE '%.parquet' THEN 'données (.parquet)'
            WHEN filename LIKE '%.metadata.json' THEN 'métadonnées (.metadata.json)'
            WHEN PARSE_FILENAME(filename) LIKE 'snap-%' THEN 'listes de manifestes (snap-*.avro)'
            ELSE 'manifestes (*.avro)' END AS sorte,
       COUNT(*) AS fichiers, ROUND(SUM(size) / 1e6, 2) AS mo
FROM read_blob('plateforme/lac/iceberg/bronze/telemetrie/**/*')
GROUP BY ALL ORDER BY ALL;
```

sorte varchar	fichiers int64	mo double
données (.parquet)	28	7.02
listes de manifestes (snap-*.avro)	28	0.07
manifestes (*.avro)	28	0.17
métadonnées (.metadata.json)	31	0.55

Les données : 28 fichiers Parquet, un par mois, rangés dans `data/mois=2023-09/` et suivants. Tout le reste décrit la table. Chaque validation écrit un fichier `.metadata.json` neuf, qui porte le schéma, la partition et la liste des instantanés : 28 ajouts, une création, une partition posée et une évolution de schéma font 31 fichiers. Chaque instantané a sa liste de manifestes, et chaque ajout son manifeste, qui nomme le fichier de données ajouté avec ses statistiques. Rien n'a été effacé : le fichier `00000` décrit encore la table vide du premier instant.

© 2026 iAlexMG. Tous droits réservés.

Étape 4 — Lire la table avec DuckDB

Théorie : `Theorie_10_DE.md`, sections 3, 4 et 5.

DuckDB 1.5 lit une table Iceberg par son fichier de métadonnées, mais ne parle pas à un catalogue SQL. Un script de quelques lignes fait le lien : il attache le catalogue, et range le chemin courant de chaque table dans une variable. Créez-le dans le dépôt, `notepad sql\lac_iceberg.sql`, depuis `01_de` :

```
-- lac_iceberg.sql : ouvre les tables Iceberg du lac dans DuckDB. Chaque variable tient le
-- chemin du fichier de métadonnées courant d'une table : relisez ce fichier après une écriture.

LOAD iceberg;
LOAD sqlite;
ATTACH IF NOT EXISTS 'plateforme/lac/iceberg/catalogue.db' AS catalogue (TYPE sqlite, READ_ONLY);

SET VARIABLE telemetrie = (SELECT metadata_location FROM catalogue.iceberg_tables
                           WHERE table_namespace = 'bronze' AND table_name = 'telemetrie');
SET VARIABLE usine_horaire = (SELECT metadata_location FROM catalogue.iceberg_tables
                              WHERE table_namespace = 'argent' AND table_name = 'usine_horaire');

SELECT table_namespace AS espace, table_name AS table_,
       REGEXP_EXTRACT(metadata_location, '[^/]+$') AS metadonnees_courantes
FROM catalogue.iceberg_tables
ORDER BY ALL;
```

La variable `usine_horaire` reste vide jusqu'à l'étape 8. Dans la ligne de commande DuckDB, lisez le fichier, puis la table :

```
.read 01_de/sql/lac_iceberg.sql
SELECT COUNT(*) AS messages, COUNT(temperature_moteur_max) AS temperature,
       COUNT(heures_moteur) AS heures_moteur, MIN(jour) AS premier_jour, MAX(jour) AS dernier_jour
FROM iceberg_scan(GETVARIABLE('telemetrie'));
```

espace varchar	table_ varchar	metadonnees_courantes varchar
bronze	telemetrie	00030-cdbc6121-0cb3-49af-9056-733187c682ab.metadata.json

messages int64	temperature int64	heures_moteur int64	premier_jour date	dernier_jour date
158146	158146	69468	2023-09-09	2025-12-31

© 2026 iAlexMG. Tous droits réservés.

Une seule colonne de température, remplie pour les 158 146 messages : ceux de 2023, écrits sous le nom `temp_moteur_max`, se lisent sous le nom neuf. Les 69 468 heures du moteur sont les messages du micrologiciel 2.0 reçus avant 2026. Le bronze demandait un `COALESCE` ; la table n'en demande plus.

Aucun fichier de 2023 ou de 2024 n'a pourtant été réécrit. Ouvrez celui de mars 2024 et regardez ses colonnes, avec l'identifiant que chacune porte :

```
SET VARIABLE fichier = (SELECT file_path FROM iceberg_metadata(GETVARIABLE('telemetrie'))
                        WHERE file_path LIKE '%mois=2024-03%');
SELECT name, field_id FROM parquet_schema(GETVARIABLE('fichier'))
WHERE name IN ('temp_moteur_max', 'temperature_moteur_max', 'heures_moteur');
```

name varchar	field_id int64
temp_moteur_max	9

Le fichier ne connaît que l'ancien nom, sous l'identifiant 9. Le fichier de métadonnées, lui, est du JSON : DuckDB le lit comme n'importe quel fichier, et on y trouve les schémas successifs de la table. `UNNEST` déplie une liste en lignes, et `LATERAL` laisse le second dépliage lire le résultat du premier : aplatir un document est au Cours 08 de la formation SQL avancé, section 6, et `LATERAL` à son Cours 07, section 7.

```
SELECT s."schema-id" AS schema, f.id AS id, f.name AS nom
FROM (SELECT UNNEST(schemas) AS s FROM read_json(GETVARIABLE('telemetrie'))),
     LATERAL (SELECT UNNEST(s.fields) AS f)
WHERE f.name LIKE 'temp%moteur%' OR f.name = 'heures_moteur'
ORDER BY ALL;
```

schema int64	id int64	nom varchar
0	9	temp_moteur_max
1	9	temperature_moteur_max
1	20	heures_moteur

Voilà tout le mécanisme. Le schéma 1 donne à l'identifiant 9 un nom neuf, et crée l'identifiant 20. Un lecteur qui demande `temperature_moteur_max` cherche la colonne 9 dans chaque fichier, quel que soit le nom qu'elle y porte ; il cherche la colonne 20 dans le fichier de 2024, ne la trouve pas, et rend `NULL`.

Reste la partition. Demandez le plan d'une requête sur octobre 2025, avec ce qu'elle a vraiment lu :

```
EXPLAIN ANALYZE
SELECT COUNT(*) FROM iceberg_scan(GETVARIABLE('telemetrie'))
WHERE jour BETWEEN DATE '2025-10-01' AND DATE '2025-10-31';
```


© 2026 iAlexMG. Tous droits réservés.

Dans la boîte `TABLE_SCAN` du plan :

```
│    Total Files Read: 1    │
│                           │
│        Filename(s):       │
…
│   /mois=2025-10/00000-0   │
…
│         8,775 rows        │
```

Un fichier ouvert sur 28. La requête filtre sur `jour`, une vraie colonne ; la table sait que sa partition est le mois de `jour`, et les manifestes gardent pour chaque fichier la plage de ses valeurs. Personne n'a écrit `mois = '2025-10'`.

Étape 5 — Les trois mois de 2026, jour par jour

Théorie : `Theorie_10_DE.md`, section 6.

Un chargement quotidien fait un instantané par jour. Reprenez là où la table s'est arrêtée, jusqu'à la fin de mars 2026, avec `--par-jour`, depuis `01_de` :

```
python -m tables.telemetrie_iceberg --jusqua 2026-03 --par-jour
```

```
90 ajouts, 46339 messages, en 45.2 s.
La table : 118 instantanés, 118 fichiers de données.
Métadonnées courantes : C:/…/Projet_Boreal_intermediaire/plateforme/lac/iceberg/bronze/telemetrie/metadata/00120-….metadata.json
```

Relancez la même commande : `0 ajouts, 0 messages`. Le chargeur repart du dernier jour que la table contient, et un second passage ne double rien.

L'historique de la table se lit par le module d'entretien, qui porte aussi le compactage, l'expiration et le nettoyage de l'étape 7. Créez-le, `notepad tables\entretien_iceberg.py`. L'en-tête, et le décompte des fichiers présents sur le disque :

```
# -*- coding: utf-8 -*-
"""entretien_iceberg.py : l'historique d'une table Iceberg, et son entretien.

Usage, depuis 01_de : python -m tables.entretien_iceberg GESTE, où GESTE est historique,
etiqueter NOM --rang N, compacter AAAA-MM..., expirer [--garder 1] ou nettoyer [--age 72]
"""
import argparse
import time
from datetime import date
from pathlib import Path

from tables.catalogue import catalogue
from tables.telemetrie_iceberg import TABLE
```


© 2026 iAlexMG. Tous droits réservés.

```
def fichiers_sur_disque(table) -> list[Path]:
    """Les fichiers Parquet présents dans le dossier de données de la table, cités ou non."""
    return list(Path(table.location()).joinpath("data").rglob("*.parquet"))
```

L'historique, un instantané par ligne, et l'étiquette :

```
def historique(table):
    etiquettes = {ref.snapshot_id: nom for nom, ref in table.refs().items() if nom != "main"}
    for rang, s in enumerate(table.snapshots(), 1):
        resume = s.summary
        quoi = resume.get("boreal.periode") or resume.get("boreal.operation") or ""
        lignes = resume.get("total-records") or ""
        print(f"{rang:4}  {s.snapshot_id:>20}  {resume.operation.value:9}  {quoi:24}  "
              f"{lignes:>8}  {etiquettes.get(s.snapshot_id, '')}")


def etiqueter(table, nom, rang):
    instantane = table.snapshots()[rang - 1]
    table.manage_snapshots().create_tag(instantane.snapshot_id, nom).commit()
    print(f"L'instantané de rang {rang} ({instantane.snapshot_id}) s'appelle maintenant {nom}.")
```

Le compactage d'un mois, par une réécriture limitée à ce mois :

```
def compacter(table, mois):
    for m in mois:
        annee, numero = map(int, m.split("-"))
        debut, fin = date(annee, numero, 1), date(annee + (numero == 12), numero % 12 + 1, 1)
        filtre = f"jour >= '{debut}' AND jour < '{fin}'"
        partition = (annee - 1970) * 12 + numero - 1     # un mois se range en mois depuis 1970-01

        def fichiers_du_mois():
            return sum(1 for f in table.inspect.files()["partition"].to_pylist()
                       if f["mois"] == partition)

        avant = fichiers_du_mois()
        lignes = table.scan(row_filter=filtre).to_arrow()
        table.overwrite(lignes, overwrite_filter=filtre,
                        snapshot_properties={"boreal.operation": f"compactage de {m}"})
```


© 2026 iAlexMG. Tous droits réservés.

```
        print(f"{m} : {lignes.num_rows} lignes, {avant} fichiers réécrits "
              f"en {fichiers_du_mois()}.")
```

L'expiration et le nettoyage :

```
def expirer(table, garder):
    proteges = {ref.snapshot_id for ref in table.refs().values()}
    anciens = [s.snapshot_id for s in table.snapshots()[:-garder] if s.snapshot_id not in proteges]
    if anciens:
        table.maintenance.expire_snapshots().by_ids(anciens).commit()
    print(f"{len(anciens)} instantanés expirés ; --garder {garder} et les étiquettes "
          "épargnent le reste.")


def nettoyer(table, age):
    cites = {Path(f).resolve() for f in table.inspect.all_files()["file_path"].to_pylist()}
    limite = time.time() - age * 3600
    orphelins = [f for f in fichiers_sur_disque(table)
                 if f.resolve() not in cites and f.stat().st_mtime < limite]
    for f in orphelins:
        f.unlink()
    print(f"{len(orphelins)} fichiers effacés, que plus rien ne cite depuis plus de {age:g} h.")
```

Et la porte d'entrée, qui finit toujours par l'état de la table :

```
def principal():
    p = argparse.ArgumentParser(description="L'entretien d'une table Iceberg du lac de Boréal.")
    p.add_argument("geste", choices=["historique", "etiqueter", "compacter", "expirer", "nettoyer"])
    p.add_argument("valeurs", nargs="*", help="le nom de l'étiquette, ou les mois à compacter")
    p.add_argument("--rang", type=int, help="pour etiqueter : le rang de l'instantané")
    p.add_argument("--garder", type=int, default=1, help="pour expirer : les derniers à garder")
    p.add_argument("--age", type=float, default=72, help="pour nettoyer : l'âge minimal, en heures")
    p.add_argument("--table", default=TABLE, help="la table du catalogue")
    a = p.parse_args()

    cat = catalogue()
    table = cat.load_table(a.table)
    if a.geste == "historique":
        historique(table)
    elif a.geste == "etiqueter":
        etiqueter(table, a.valeurs[0], a.rang)
```


© 2026 iAlexMG. Tous droits réservés.

```
    elif a.geste == "compacter":
        compacter(table, a.valeurs)
    elif a.geste == "expirer":
        expirer(table, a.garder)
    else:
        nettoyer(table, a.age)
    table = cat.load_table(a.table)
    print(f"La table : {len(table.snapshots())} instantanés, {table.inspect.files().num_rows} "
          f"fichiers de données cités, {len(fichiers_sur_disque(table))} sur le disque.")


if __name__ == "__main__":
    principal()
```

Affichez l'historique :

```
python -m tables.entretien_iceberg historique
```

```
   1   4363632803220730451  append     2023-09-01 au 2023-09-30        53
   2   4213760071734566944  append     2023-10-01 au 2023-10-31       253
   3   1418684707901099528  append     2023-11-01 au 2023-11-30       615
   4    571422395783015316  append     2023-12-01 au 2023-12-31      1384
…
  27   7575668681480743974  append     2025-11-01 au 2025-11-30    148760
  28   5089595928198827308  append     2025-12-01 au 2025-12-31    158146
  29   5157792201729973441  append     2026-01-01                  158971
  30   5202220848332114016  append     2026-01-02                  159360
  31   2771212747944889026  append     2026-01-03                  160215
…
 116     47844550937683676  append     2026-03-29                  204199
 117    163732998468236429  append     2026-03-30                  204349
 118   6926834923005715021  append     2026-03-31                  204485
La table : 118 instantanés, 118 fichiers de données cités, 118 sur le disque.
```

Une ligne par instantané : son rang, son identifiant, tiré au hasard et autre chez vous, son opération, la période chargée, et le nombre de lignes de la table à ce moment-là. Le rang 28 est l'état au 31 décembre 2025. Donnez-lui un nom, pour le retrouver sans son identifiant :

```
python -m tables.entretien_iceberg etiqueter fin-2025 --rang 28
```

```
L'instantané de rang 28 (5089595928198827308) s'appelle maintenant fin-2025.
La table : 118 instantanés, 118 fichiers de données cités, 118 sur le disque.
```

Étape 6 — Remonter au 31 décembre 2025

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_10_DE.md`, section 6.

Dans la ligne de commande DuckDB, relisez d'abord `lac_iceberg.sql` : la variable `telemetrie` pointe encore vers le fichier `00030`, écrit avant 2026, et une requête sur elle lirait la table d'avant l'étape 5. C'est déjà un voyage dans le temps, involontaire.

```
.read 01_de/sql/lac_iceberg.sql
SELECT COUNT(*) AS messages FROM iceberg_scan(GETVARIABLE('telemetrie'));
SELECT refs FROM read_json(GETVARIABLE('telemetrie'));
```

espace varchar	table_ varchar	metadonnees_courantes varchar
bronze	telemetrie	00121-8b83fced-897f-4e1c-aaab-9641bef21939.metadata.json

messages int64
204485

refs struct(main struct("snapshot-id" bigint, "type" varchar), "fin-2025" struct("snapshot-id" bigint, "type" varchar))
{'main': {'snapshot-id': 6926834923005715021, 'type': branch}, 'fin-2025': {'snapshot-id': 5089595928198827308, 'type': tag}}

La table compte ses 204 485 messages. Ses références sont deux : `main`, la branche courante, et l'étiquette `fin-2025`, chacune pointant vers un instantané. Rangez celui de l'étiquette dans une variable, et lisez la table telle qu'elle était :

```
SET VARIABLE fin_2025 = (SELECT refs['fin-2025']['snapshot-id']
                         FROM read_json(GETVARIABLE('telemetrie')));
SELECT COUNT(*) AS messages, MAX(jour) AS dernier_jour
FROM iceberg_scan(GETVARIABLE('telemetrie'), snapshot_from_id => GETVARIABLE('fin_2025'));
```

messages int64	dernier_jour date
158146	2025-12-31

© 2026 iAlexMG. Tous droits réservés.

Le compte ne prouve pas le contenu. Comparez l'instantané au bronze arrêté à la même date, message par message, dans les deux sens. `EXCEPT ALL` garde les doublons, que le bronze contient :

```
WITH instantane AS (
    SELECT message_id, recu_le, sujet
    FROM iceberg_scan(GETVARIABLE('telemetrie'), snapshot_from_id => GETVARIABLE('fin_2025'))),
bronze AS (
    SELECT message.message_id AS message_id, recu_le, sujet
    FROM read_parquet('plateforme/lac/bronze/telemetrie/**/*.parquet', hive_partitioning = true)
    WHERE jour <= DATE '2025-12-31')
SELECT (SELECT COUNT(*) FROM (FROM instantane EXCEPT ALL FROM bronze)) AS en_trop,
       (SELECT COUNT(*) FROM (FROM bronze EXCEPT ALL FROM instantane)) AS manquants;
```

en_trop	manquants
int64	int64
0	0

Rien en trop, rien qui manque : l'étiquette rend l'état au 31 décembre. Essayez maintenant de remonter par la date, comme on le ferait d'instinct :

```
SELECT COUNT(*) AS messages
FROM iceberg_scan(GETVARIABLE('telemetrie'),
                  snapshot_from_timestamp => TIMESTAMP '2025-12-31 23:59:59');
```

messages
int64
0

Zéro, et aucune erreur. Le voyage dans le temps suit l'heure des validations, pas celle des données : au 31 décembre 2025, la table n'existait pas encore. Regardez quand ses instantanés ont été écrits :

```
SELECT sequence_number, snapshot_id, timestamp_ms
FROM iceberg_snapshots(GETVARIABLE('telemetrie'))
WHERE sequence_number IN (1, 28, 29, 118);
```

sequence_number	snapshot_id	timestamp_ms
uint64	uint64	timestamp

© 2026 iAlexMG. Tous droits réservés.

```
┌──────────────┬─────────────────────┬─────────────────────────┐
│              1 │ 4363632803220730451 │ 2026-09-23 11:19:41.176 │
│             28 │ 5089595928198827308 │ 2026-09-23 11:19:54.166 │
│             29 │ 5157792201729973441 │ 2026-09-23 11:19:57.231 │
│            118 │ 6926834923005715021 │ 2026-09-23 11:20:41.937 │
└──────────────┴─────────────────────┴─────────────────────────┘
```

Quatre-vingt-dix jours de 2026 écrits en moins d'une minute, en septembre 2026, à l'heure UTC. Les dates des données vivent dans les données, et dans la période que chaque instantané porte ; l'horloge des validations ne sert qu'à dire ce que la table contenait à tel moment de sa propre vie.

Étape 7 — Compacter, expirer, nettoyer

Théorie : `Theorie_10_DE.md`, section 7.

Quatre-vingt-dix ajouts ont laissé 90 petits fichiers pour trois mois. Mesurez une lecture de 2026 avant d'y toucher, trois fois de suite pour que les caches se remplissent :

```
.timer on
SELECT COUNT(*), AVG(distance_km) FROM iceberg_scan(GETVARIABLE('telemetrie'))
WHERE jour >= DATE '2026-01-01';
```

```
┌──────────┬──────────┬──────────────────────────────────────────────────────────────────┐
│ espace   │  table_  │                    metadonnees_courantes                         │
│ varchar  │ varchar  │                            varchar                               │
├──────────┼──────────┼──────────────────────────────────────────────────────────────────┤
│ bronze   │ telemetrie │ 00121-8b83fced-897f-4e1c-aaab-9641bef21939.metadata.json      │
└──────────┴──────────┴──────────────────────────────────────────────────────────────────┘

┌──────────────┬──────────────────┐
│ count_star() │ avg(distance_km) │
│    int64     │      double      │
├──────────────┼──────────────────┤
│        46339 │ 47.563044088133026 │
└──────────────┴──────────────────┘
Run Time (s): real 0.081 user 0.296875 sys 0.093750
Run Time (s): real 0.078 user 0.390625 sys 0.093750
Run Time (s): real 0.068 user 0.421875 sys 0.062500
```

Compactez les trois mois, depuis `01_de` :

```
python -m tables.entretien_iceberg compacter 2026-01 2026-02 2026-03
```

```
2026-01 : 16076 lignes, 31 fichiers réécrits en 1.
2026-02 : 17149 lignes, 28 fichiers réécrits en 1.
```


© 2026 iAlexMG. Tous droits réservés.

```
2026-03 : 13114 lignes, 31 fichiers réécrits en 1.
La table : 124 instantanés, 31 fichiers de données cités, 121 sur le disque.
```

Trente et un fichiers cités : 28 mois et 3 mois compactés. Mais 121 sur le disque, parce que le compactage n'efface rien : les 90 fichiers quotidiens appartiennent encore aux instantanés 29 à 118. Et six instantanés de plus, deux par mois, puisqu'une réécriture en fait deux : l'un retire les fichiers du mois, l'autre ajoute le fichier neuf.

```
…
 118   6926834923005715021  append     2026-03-31                 204485
 119   2123974747198056148  delete     compactage de 2026-01      188409
 120   6643334449157534903  append     compactage de 2026-01      204485
 121   8627860494312612510  delete     compactage de 2026-02      187336
 122   7601200391776581225  append     compactage de 2026-02      204485
 123   3294025430216378304  delete     compactage de 2026-03      191371
 124   4028751766109107204  append     compactage de 2026-03      204485
La table : 124 instantanés, 31 fichiers de données cités, 121 sur le disque.
```

Relisez `lac_iceberg.sql` et refaites la mesure :

espace varchar	table_ varchar	metadonnees_courantes varchar
bronze	telemetrie	00124-88ed2ffc-18f0-4af3-a8af-c9dcea123104.metadata.json

count_star() int64	avg(distance_km) double
46339	47.56304408813335

```
Run Time (s): real 0.052 user 0.125000 sys 0.093750
Run Time (s): real 0.031 user 0.125000 sys 0.031250
Run Time (s): real 0.033 user 0.125000 sys 0.015625
```

Deux fois plus rapide, sur une lecture qui restait courte : la table est petite, et le gain grandit avec le nombre de fichiers. La moyenne bouge dans ses derniers chiffres, d'une lecture à l'autre et d'un rangement à l'autre, parce que les additions se font dans un autre ordre, comme au Cours 07 entre DuckDB et Spark. Quittez le chronomètre par `.timer off`.

Les 90 fichiers quotidiens ne servent plus qu'aux instantanés 29 à 118. Expirez tous les instantanés, sauf le dernier et ceux qui portent une étiquette :

```
python -m tables.entretien_iceberg expirer --garder 1
python -m tables.entretien_iceberg historique
```


© 2026 iAlexMG. Tous droits réservés.

```
122 instantanés expirés ; --garder 1 et les étiquettes épargnent le reste.
La table : 2 instantanés, 31 fichiers de données cités, 121 sur le disque.
   1   5089595928198827308  append     2025-12-01 au 2025-12-31    158146  fin-2025
   2   4028751766109107204  append     compactage de 2026-03       204485
La table : 2 instantanés, 31 fichiers de données cités, 121 sur le disque.
```

Deux instantanés restent : l'étiqueté et le courant. Dans la ligne de commande DuckDB, relisez `lac_iceberg.sql`, puis l'étiquette, puis l'instantané de rang 29, dont l'identifiant figure dans l'historique de l'étape 5 (remplacez le nombre par le vôtre) :

```
.read 01_de/sql/lac_iceberg.sql
SET VARIABLE fin_2025 = (SELECT refs['fin-2025']['snapshot-id']
                         FROM read_json(GETVARIABLE('telemetrie')));
SELECT COUNT(*) AS messages
FROM iceberg_scan(GETVARIABLE('telemetrie'), snapshot_from_id => GETVARIABLE('fin_2025'));
SELECT COUNT(*) AS messages
FROM iceberg_scan(GETVARIABLE('telemetrie'), snapshot_from_id => 5157792201729973441);
```

espace varchar	table_ varchar	metadonnees_courantes varchar
bronze	telemetrie	00125-4a40456d-d938-4e22-b915-69a04b084440.metadata.json

messages int64
158146

```
Invalid Configuration Error: Could not find snapshot with id 5157792201729973441
```

L'étiquette se relit : 158 146 messages. Le 1er janvier 2026, lui, est perdu pour le voyage dans le temps. Ses fichiers sont pourtant toujours sur le disque, 121 en tout : PyIceberg expire les instantanés sans effacer ce qu'ils étaient seuls à citer. C'est le rôle du nettoyage, qui efface les fichiers que plus aucun instantané ne cite :

```
python -m tables.entretien_iceberg nettoyer
python -m tables.entretien_iceberg nettoyer --age 0
```

```
0 fichiers effacés, que plus rien ne cite depuis plus de 72 h.
La table : 2 instantanés, 31 fichiers de données cités, 121 sur le disque.
90 fichiers effacés, que plus rien ne cite depuis plus de 0 h.
La table : 2 instantanés, 31 fichiers de données cités, 31 sur le disque.
```


© 2026 iAlexMG. Tous droits réservés.

Le premier passage n'efface rien : par défaut, le nettoyage épargne tout fichier de moins de trois jours, qui pourrait appartenir à une écriture pas encore validée. Le second, `--age 0`, est permis ici parce que personne d'autre n'écrit dans cette table. Refaites le décompte de l'étape 3 :

sorte varchar	fichiers int64	mo double
données (.parquet)	31	9.04
listes de manifestes (snap-*.avro)	124	0.6
manifestes (*.avro)	124	0.81
métadonnées (.metadata.json)	126	8.49

Les données sont revenues à 31 fichiers. Les métadonnées, elles, n'ont rien perdu : un fichier `.metadata.json` par écriture depuis la création de la table, 126 en tout, qui recopie chaque fois la liste des instantanés. Avec les manifestes, elles pèsent maintenant plus lourd que les données, parce que le nettoyage du cours ne touche qu'aux données. La théorie dit comment une table borne ses métadonnées.

Étape 8 — Les séries de l'usine en argent

Théorie : `Theorie_10_DE.md`, sections 5, 9 et 10.

Le bronze garde la télémétrie telle que reçue. L'argent garde ce que Data Engineering a nettoyé : les trois séries de l'usine du Cours 08, qui vivent dans le schéma `argent` de l'entrepôt. Mettez-les en tables Iceberg, `notepad tables\argent_iceberg.py` :

```
# -*- coding: utf-8 -*-
"""argent_iceberg.py : les séries de l'usine, relues de l'entrepôt, en tables Iceberg.

Usage, depuis 01_de : python -m tables.argent_iceberg
"""
import time

import duckdb
from pyiceberg.exceptions import NoSuchTableError
from pyiceberg.transforms import MonthTransform

from pipeline.config import chemin
from tables.catalogue import catalogue

SERIES = ["usine_horaire", "production_horaire", "energie_horaire"]


def principal():
```


© 2026 iAlexMG. Tous droits réservés.

```
    cat = catalogue()
    con = duckdb.connect(str(chemin("entrepot")), read_only=True)
    # Iceberg ne range un horodatage avec fuseau qu'en UTC : exporté à l'heure de l'Est, il serait
    # refusé (« unsupported type: timestamp[us, tz=America/Toronto] »). L'instant ne change pas.
    con.sql("SET TimeZone = 'UTC'")
    t0 = time.time()
    for nom in SERIES:
        donnees = con.sql(f"SELECT * FROM argent.{nom} ORDER BY heure").to_arrow_table()
        try:
            table = cat.load_table(f"argent.{nom}")
        except NoSuchTableError:
            table = cat.create_table(f"argent.{nom}", schema=donnees.schema)
            with table.update_spec() as spec:
                spec.add_field("heure", MonthTransform(), "mois")
        if table.current_snapshot() is None:
            table.append(donnees)          # une table neuve : rien à remplacer
        else:
            table.overwrite(donnees)       # deux instantanés : l'un efface, l'autre écrit
        table = cat.load_table(f"argent.{nom}")
        print(f"argent.{nom} : {donnees.num_rows} lignes, "
              f"{table.inspect.files().num_rows} fichiers, {len(table.snapshots())} instantanés.")
    con.close()
    print(f"En {time.time() - t0:.1f} s.")


if __name__ == "__main__":
    principal()
```

Le programme ouvre l'entrepôt en lecture seule : la ligne de commande DuckDB peut rester ouverte, tant qu'elle n'a pas attaché l'entrepôt en écriture. Lancez-le deux fois, depuis `01_de` :

```
python -m tables.argent_iceberg
python -m tables.argent_iceberg
```

```
argent.usine_horaire : 240240 lignes, 7 fichiers, 1 instantanés.
argent.production_horaire : 13104 lignes, 7 fichiers, 1 instantanés.
argent.energie_horaire : 37223 lignes, 52 fichiers, 1 instantanés.
En 0.7 s.
argent.usine_horaire : 240240 lignes, 7 fichiers, 3 instantanés.
argent.production_horaire : 13104 lignes, 7 fichiers, 3 instantanés.
argent.energie_horaire : 37223 lignes, 52 fichiers, 3 instantanés.
En 0.8 s.
```


© 2026 iAlexMG. Tous droits réservés.

Le second passage remplace chaque table sans rien doubler, et laisse l'état d'avant dans l'historique : deux instantanés de plus, l'un qui efface, l'autre qui écrit. Comparez `usine_horaire` à sa source, dans la ligne de commande DuckDB :

```
.read 01_de/sql/lac_iceberg.sql
ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot (READ_ONLY);
SELECT COUNT(*) AS lignes, MIN(heure) AS premiere, MAX(heure) AS derniere
FROM iceberg_scan(GETVARIABLE('usine_horaire'));
SELECT (SELECT COUNT(*) FROM (FROM iceberg_scan(GETVARIABLE('usine_horaire'))
                              EXCEPT ALL FROM entrepot.argent.usine_horaire)) AS en_trop,
       (SELECT COUNT(*) FROM (FROM entrepot.argent.usine_horaire
                              EXCEPT ALL FROM iceberg_scan(GETVARIABLE('usine_horaire')))) AS manquants;
```

espace varchar	table_ varchar	metadonnees_courantes varchar
argent	energie_horaire	00003-f1108da5-3c26-4271-94e1-a95d5dd0a5cf.metadata.json
argent	production_horaire	00003-7f6c3ee9-1c72-455d-9ccf-b9d9bf706190.metadata.json
argent	usine_horaire	00003-cdb489b3-0b0a-4390-8132-428ea16eb1d0.metadata.json
bronze	telemetrie	00125-4a40456d-d938-4e22-b915-69a04b084440.metadata.json

lignes int64	premiere timestamp with time zone	derniere timestamp with time zone
240240	2025-10-01 00:00:00-04	2026-03-31 23:00:00-04

en_trop int64	manquants int64
0	0

Les 240 240 lignes, 55 étiquettes pendant 4 368 heures, sont identiques des deux côtés. Deux détails de traduction méritent un coup d'œil. Le premier est un type :

```
SELECT i.column_name, i.column_type AS dans_iceberg, e.column_type AS dans_l_entrepot
FROM (DESCRIBE SELECT * FROM iceberg_scan(GETVARIABLE('usine_horaire'))) i
JOIN (DESCRIBE entrepot.argent.usine_horaire) e USING (column_name)
WHERE i.column_type <> e.column_type;
```


© 2026 iAlexMG. Tous droits réservés.

column_name varchar	dans_iceberg varchar	dans_l_entrepot varchar
couverture_s	DECIMAL(38,0)	HUGEINT

`HUGEINT`, l'entier de 128 bits de DuckDB, n'existe pas dans Iceberg : il sort en décimal de 38 chiffres. Les valeurs sont les mêmes, la comparaison le prouve, mais un autre moteur verra un décimal. Le second détail est une partition de trop :

```
dir /b ..\plateforme\lac\iceberg\argent\usine_horaire\data
```

```
mois=2025-10
mois=2025-11
mois=2025-12
mois=2026-01
mois=2026-02
mois=2026-03
mois=2026-04
```

Sept mois pour des séries qui s'arrêtent le 31 mars. Le mois d'une partition se compte en UTC, et les heures de 20 h à 23 h le 31 mars, heure de l'Est, sont déjà le 1er avril à Greenwich :

```
SELECT COUNT(*) AS lignes
FROM iceberg_scan(GETVARIABLE('usine_horaire'))
WHERE heure >= TIMESTAMPTZ '2026-03-31 20:00:00-04';
```

lignes int64
220

Quatre heures de 55 étiquettes. Rien de faux : une requête sur mars filtre sur `heure` et trouve ses lignes dans deux partitions. Mais qui compte les partitions pour estimer un volume sera surpris.

Étape 9 — La même télémétrie en Delta Lake

Théorie : `Theorie_10_DE.md`, sections 2, 4, 7 et 8.

Delta Lake range la même idée autrement : un journal de validations, `_delta_log/`, à côté des fichiers de données. Le chargeur réemploie la lecture et le découpage du chargeur Iceberg, et écrit un mois par version. Créez-le, `notepad tables\telemetrie_delta.py`. Le chargement et l'historique :

© 2026 iAlexMG. Tous droits réservés.

```python
# -*- coding: utf-8 -*-
"""telemetrie_delta.py : la même télémétrie en table Delta Lake, pour comparer.

Usage, depuis 01_de : python -m tables.telemetrie_delta [--historique | --compacter | --nettoyer]
"""
import argparse
import time

import duckdb
from deltalake import CommitProperties, DeltaTable, write_deltalake

from pipeline.config import chemin
from tables.telemetrie_iceberg import DEBUT, MICROLOGICIEL_2, lire, nommer, periodes


def charger(dossier):
    con = duckdb.connect()
    bronze = (chemin("lac") / "bronze" / "telemetrie").as_posix()
    con.sql(f"CREATE VIEW bronze AS SELECT * FROM read_parquet('{bronze}/**/*.parquet', "
            "hive_partitioning = true)")
    dernier = con.sql("SELECT max(jour) FROM bronze").fetchone()[0]
    t0, versions = time.time(), 0
    for debut, fin in periodes(DEBUT, dernier, par_jour=False):
        deux = debut >= MICROLOGICIEL_2
        donnees = lire(con, debut, fin, deux)
        if donnees.num_rows == 0:
            continue
        # schema_mode="merge" : une colonne inconnue de la table s'y ajoute au lieu d'être refusée
        write_deltalake(dossier, donnees, mode="append", schema_mode="merge" if deux else None,
                        commit_properties=CommitProperties(
                            custom_metadata={"boreal.periode": nommer(debut, fin)}))
        versions += 1
    print(f"{versions} écritures en {time.time() - t0:.1f} s.")


def historique(table):
    for version in reversed(table.history()):
        mesures = version.get("operationMetrics", {})
        lignes = mesures.get("num_added_rows", "")
        print(f"{version['version']:4}  {version['operation']:12}  "
              f"{version.get('boreal.periode', ''):24}  {lignes:>8}")
```


© 2026 iAlexMG. Tous droits réservés.

Puis la porte d'entrée, avec le compactage et le nettoyage :

```
def principal():
    p = argparse.ArgumentParser(description="La télémétrie du bronze en table Delta.")
    p.add_argument("--historique", action="store_true", help="afficher les versions")
    p.add_argument("--compacter", action="store_true", help="réunir les petits fichiers")
    p.add_argument("--nettoyer", action="store_true",
                   help="effacer les fichiers que la version courante ne cite plus")
    a = p.parse_args()
    dossier = chemin("delta") / "telemetrie"
    if a.historique:
        historique(DeltaTable(dossier))
    elif a.compacter:
        mesures = DeltaTable(dossier).optimize.compact()
        print(f"Compactage : {mesures['numFilesRemoved']} fichiers réunis "
              f"en {mesures['numFilesAdded']}.")
    elif a.nettoyer:
        # Par défaut, delta-rs refuse d'effacer ce qui a moins de 7 jours : c'est la durée pendant
        # laquelle une lecture en cours peut encore avoir besoin d'un vieux fichier.
        effaces = DeltaTable(dossier).vacuum(retention_hours=0, enforce_retention_duration=False,
                                             dry_run=False)
        print(f"Nettoyage : {len(effaces)} fichiers effacés.")
    elif dossier.exists():
        print(f"{dossier} existe déjà : effacez-le pour recharger.")
    else:
        charger(dossier)
    table = DeltaTable(dossier)
    colonnes = [c.name for c in table.schema().fields]
    moteur = [c for c in colonnes if "moteur" in c]
    print(f"La table : version {table.version()}, {len(table.file_uris())} fichiers, "
          f"{len(colonnes)} colonnes, dont {', '.join(moteur)}.")


if __name__ == "__main__":
    principal()
```

`schema_mode="merge"` est la seule évolution que la bibliothèque permet ici : une colonne inconnue s'ajoute à la table. Chargez, depuis `01_de`, puis affichez l'historique :

```
python -m tables.telemetrie_delta
python -m tables.telemetrie_delta --historique
```


© 2026 iAlexMG. Tous droits réservés.

```
31 écritures en 14.0 s.
La table : version 30, 31 fichiers, 20 colonnes, dont temp_moteur_max, heures_moteur, temperature_moteur_max.
   0  WRITE         2023-09-01 au 2023-09-30        53
   1  WRITE         2023-10-01 au 2023-10-31       200
   2  WRITE         2023-11-01 au 2023-11-30       362
…
  25  WRITE         2025-10-01 au 2025-10-31      8775
  26  WRITE         2025-11-01 au 2025-11-30      7386
  27  WRITE         2025-12-01 au 2025-12-31      9386
  28  WRITE         2026-01-01 au 2026-01-31     16076
  29  WRITE         2026-02-01 au 2026-02-28     17149
  30  WRITE         2026-03-01 au 2026-03-31     13114
```

Trente et un mois, trente et une versions, numérotées de 0 à 30 ; la version 27 est décembre 2025. Le nombre de la dernière colonne est celui des lignes ajoutées par la version, et non le total de la table comme dans l'historique d'Iceberg. Les colonnes du moteur disent l'essentiel : la table a gardé `temp_moteur_max`, et ajouté `heures_moteur` et `temperature_moteur_max` au bout. Lisez-la dans la ligne de commande DuckDB :

```
LOAD delta;
SELECT COUNT(*) AS messages, COUNT(temp_moteur_max) AS temp_moteur_max,
       COUNT(temperature_moteur_max) AS temperature_moteur_max,
       COUNT(COALESCE(temperature_moteur_max, temp_moteur_max)) AS l_une_ou_l_autre
FROM delta_scan('plateforme/lac/delta/telemetrie');
SELECT COUNT(*) AS messages, MAX(jour) AS dernier_jour
FROM delta_scan('plateforme/lac/delta/telemetrie', version => 27);
```

messages int64	temp_moteur_max int64	temperature_moteur_max int64	l_une_ou_l_autre int64
204485	78843	125642	204485

messages int64	dernier_jour date
158146	2025-12-31

Le voyage dans le temps marche de la même façon, par un numéro de version. L'évolution du schéma, non : on retrouve les deux colonnes du bronze, et le `COALESCE` qui va avec. 78 843 messages d'avant mai 2025 d'un côté, 125 642 de l'autre, dont les 9 835 des véhicules restés en 1.x après le 1er mai.

Le journal est fait de fichiers JSON, un par version. Regardez les actions que porte la version 20, celle de mai 2025 :

© 2026 iAlexMG. Tous droits réservés.

```
SELECT JSON_KEYS(json) AS actions
FROM read_json_objects('plateforme/lac/delta/telemetrie/_delta_log/00000000000000000020.json');
```

```
┌──────────────┐
│   actions    │
│  varchar[]   │
├──────────────┤
│ [commitInfo] │
│ [metaData]   │
│ [add]        │
└──────────────┘
```

Trois lignes : qui a validé et quand, avec la période que le chargeur y a mise ; le schéma neuf, puisque cette version l'a changé ; et le fichier ajouté, avec ses statistiques. Une version ordinaire n'a que la première et la dernière.

Tentez maintenant ce qu'Iceberg a fait sans broncher. Le renommage d'une colonne Delta passe par la correspondance des colonnes par nom, qu'il faut d'abord activer, depuis `01_de` :

```
python -c "from deltalake import DeltaTable;
DeltaTable('../plateforme/lac/delta/telemetrie').alter.set_table_properties({'delta.columnMapping.mode': 'name'})"
```

```
_internal.DeltaError: Column mapping is not supported for write operation 'SET TBLPROPERTIES delta.columnMapping.mode' yet
```

La dernière ligne de la trace suffit : `deltalake` 1.6 ne sait pas activer ce mode. Le format le permet, Databricks l'active (voir l'annexe) ; la bibliothèque, pas encore. C'est une limite de l'outil, pas du format, et elle décide pourtant de ce que vous pouvez faire sur ce poste.

Finissez par l'entretien, qui suit le même ordre qu'Iceberg :

```
python -m tables.telemetrie_delta --compacter
python -m tables.telemetrie_delta --nettoyer
python -m tables.telemetrie_delta --historique
```

```
Compactage : 31 fichiers réunis en 1.
La table : version 31, 1 fichiers, 20 colonnes, dont temp_moteur_max, heures_moteur, temperature_moteur_max.
Nettoyage : 31 fichiers effacés.
La table : version 33, 1 fichiers, 20 colonnes, dont temp_moteur_max, heures_moteur, temperature_moteur_max.
…
  29  WRITE         2026-02-01 au 2026-02-28     17149
  30  WRITE         2026-03-01 au 2026-03-31     13114
  31  OPTIMIZE
  32  VACUUM START
  33  VACUUM END
La table : version 33, 1 fichiers, 20 colonnes, dont temp_moteur_max, heures_moteur, temperature_moteur_max.
```


© 2026 iAlexMG. Tous droits réservés.

Le compactage réunit les 31 fichiers en un seul, la version 31 ; le nettoyage efface les 31 anciens en deux versions, son début et sa fin. Relisez maintenant la version 27, dans la ligne de commande DuckDB :

```
SELECT COUNT(*) AS messages FROM delta_scan('plateforme/lac/delta/telemetrie', version => 27);
```

```
IO Error: Cannot open file "plateforme/lac/delta/telemetrie/part-00000-….snappy.parquet": Le fichier spécifié est introuvable.
```

Le journal sait encore ce qu'était la version 27, mais ses fichiers ont disparu. Delta n'a pas d'étiquette qui protège une version : le nettoyage garde ce que la rétention couvre, sept jours par défaut, et rien d'autre. La version courante, elle, se lit toujours : 204 485 messages.

Étape 10 — Versionner

Depuis `01_de` :

```
git add config.toml sql/lac_iceberg.sql
git add tables/__init__.py tables/catalogue.py tables/telemetrie_iceberg.py
git add tables/entretien_iceberg.py tables/argent_iceberg.py tables/telemetrie_delta.py
git commit -m "Cours 10 : les tables Iceberg et Delta du lac, et leur entretien"
```

Le lac ne se versionne pas : `plateforme/lac/` est hors du dépôt, et les tables se reconstruisent à partir du bronze et de l'entrepôt, en quelques minutes, par les commandes de ce labo.

Récapitulatif

Ce que vous avez mesuré	Le chiffre
Le bronze de la télémétrie	933 fichiers, 204 485 messages, deux colonnes pour une température
La table Iceberg au 31 décembre 2025	28 instantanés, 28 fichiers, 158 146 messages, 31 fichiers de métadonnées
L'évolution du schéma	la colonne 9 renommée, la colonne 20 ajoutée, aucun fichier réécrit
Le partitionnement caché	1 fichier lu sur 28 pour octobre 2025
Les trois mois de 2026, jour par jour	90 instantanés de plus, 46 339 messages
Le voyage dans le temps	158 146 messages à l'étiquette, 0 écart dans les deux sens ; 0 par la date
Le compactage	90 fichiers en 3, une lecture deux fois plus rapide
L'expiration et le nettoyage	122 instantanés expirés, 90 fichiers effacés, l'étiquette toujours lisible

© 2026 iAlexMG. Tous droits réservés.

Ce que vous avez mesuré	Le chiffre
L'argent en Iceberg	240 240 lignes, 0 écart avec l'entrepôt, une partition d'avril en UTC
Delta Lake	31 versions, 78 843 et 125 642 messages dans deux colonnes, le renommage refusé
Delta après son nettoyage	la version 27 illisible, la courante intacte

Ce que vous savez maintenant et que vous ne saviez pas : une table ouverte, ce sont des fichiers Parquet plus des métadonnées qui disent lesquels comptent ; renommer une colonne ne coûte rien quand la table désigne ses colonnes par un identifiant ; le voyage dans le temps suit l'horloge des validations, et l'entretien en fixe la limite ; et deux formats qui se ressemblent peuvent diverger sur ce qui vous importe, parce que la bibliothèque qui les écrit n'en fait pas toujours autant que le format.

Dépannage

Symptôme	Cause	Ce qu'on fait
`ModuleNotFoundError: No module named 'pyiceberg'` (ou `deltalake`)	les bibliothèques du cours ne sont pas installées dans `formations`	`pip install -r ..\01_Data_Engineering\requirements.txt`, depuis `Projet_Boreal_intermediaire`
`ModuleNotFoundError: No module named 'tables'`	le programme n'est pas lancé depuis `01_de`, ou `tables\__init__.py` manque	`cd` vers `Projet_Boreal_intermediaire\01_de`, puis `python -m tables.telemetrie_iceberg …`
`KeyError: 'iceberg'` au lancement	`config.toml` n'a pas les deux lignes de l'étape 2	les ajouter à la section `[chemins]`
`NotInstalledError: pyiceberg_core needs to be installed.` au premier ajout	le module compilé qui calcule les partitions manque	`pip install "pyiceberg[pyiceberg-core]"`
`La syntaxe du nom de fichier, de répertoire ou de volume est incorrecte`	un entrepôt donné par une adresse `file:///C:/…`, que PyIceberg change en `/C:/…`	garder le chemin de `catalogue.py`, écrit par `as_posix()`, sans `file:`
`FileNotFoundError: [WinError 3] Failed to open local file '…'`, à la création de la table ou à un ajout	le chemin complet d'un fichier de la table dépasse 260 caractères, la limite de Windows : le projet est rangé trop profond	rapprocher `Data_Domain` de la racine du disque, ou activer les chemins longs de Windows, ce qui demande les droits d'administrateur
`unsupported type: timestamp[us, tz=America/Toronto]`	un horodatage avec fuseau exporté à l'heure de l'Est : Iceberg ne le range qu'en UTC	la ligne `SET TimeZone = 'UTC'` de `argent_iceberg.py`

© 2026 iAlexMG. Tous droits réservés.

Symptôme	Cause	Ce qu'on fait
`IO Error: Cannot open database … catalogue.db` dans DuckDB	la ligne de commande n'est pas lancée depuis `Projet_Boreal_intermediaire`	la relancer depuis là : les chemins sont relatifs
`Binder Error: Table function cannot contain subqueries`	un `iceberg_scan((SELECT …))` : une fonction de table n'accepte qu'une valeur fixe	passer par `SET VARIABLE`, comme `lac_iceberg.sql`
Une requête sur la table ne voit pas les derniers ajouts	la variable a été posée avant l'écriture, et pointe vers d'anciennes métadonnées	`.read 01_de/sql/lac_iceberg.sql` de nouveau
`Invalid Configuration Error: Could not find snapshot with id …`	l'instantané a été expiré, ou l'identifiant vient d'une autre table	lire `historique` : seuls les instantanés qu'il liste se relisent
`telemetrie existe déjà : effacez-le pour recharger.`	la table Delta a déjà été chargée	`rmdir /s /q ..\plateforme\lac\delta\telemetrie`, depuis `01_de`, puis relancer

---

Annexe — Sur Snowflake

Pour qui veut voir une table Iceberg gérée par Snowflake. Rien de cette annexe n'a été lancé à l'écriture du cours : les faits viennent de la documentation de Snowflake, consultée le 2026-09-23, et les commandes se tapent dans une feuille de calcul de Snowsight, sur votre propre compte.

Snowflake tient ses tables Iceberg de deux façons. Soit Snowflake est le catalogue, et la table se lit et s'écrit comme une table ordinaire, voyage dans le temps compris. Soit un autre catalogue la tient, et Snowflake la lit avec moins de fonctions. Depuis le 1er juin 2026, une table dont Snowflake est le catalogue peut aussi laisser Snowflake ranger ses fichiers, sans stockage à configurer : c'est le volume `SNOWFLAKE_MANAGED`, que la documentation recommande, offert dans les régions commerciales d'AWS et d'Azure. Les autres moteurs lisent alors la table par le catalogue Horizon de Snowflake. Un volume externe, lui, pointe vers votre propre stockage objet (Amazon S3, Google Cloud Storage, Azure).

Préparez deux fichiers Parquet à plat, l'un sous l'ancien schéma et l'autre sous le nouveau, dans la ligne de commande DuckDB, depuis `Projet_Boreal_intermediaire`, la table Iceberg de l'étape 7 ouverte :

```
.read 01_de/sql/lac_iceberg.sql
COPY (SELECT message_id, sernr, micrologiciel, distance_km,
             temperature_moteur_max AS temp_moteur_max, jour
      FROM iceberg_scan(GETVARIABLE('telemetrie')) WHERE jour < DATE '2025-05-01')
TO 'telemetrie_v1.parquet';
COPY (SELECT message_id, sernr, micrologiciel, distance_km,
             temperature_moteur_max, heures_moteur, jour
```


© 2026 iAlexMG. Tous droits réservés.

```
      FROM iceberg_scan(GETVARIABLE('telemetrie')) WHERE jour >= DATE '2025-05-01')
TO 'telemetrie_v2.parquet';
SELECT (SELECT COUNT(*) FROM 'telemetrie_v1.parquet') AS v1,
       (SELECT COUNT(*) FROM 'telemetrie_v2.parquet') AS v2;
```

v1 int64	v2 int64
78843	125642

Le premier fichier porte les 78 843 messages d'avant le micrologiciel 2.0, sous l'ancien nom ; le second les 125 642 suivants. Six ou sept colonnes suffisent à montrer le mécanisme.

Dans Snowsight, créez la base, un stage pour déposer les fichiers, et la table, partitionnée par mois comme au labo :

```
USE ROLE SYSADMIN;
CREATE DATABASE IF NOT EXISTS BOREAL;
CREATE SCHEMA IF NOT EXISTS BOREAL.BRONZE;
CREATE STAGE IF NOT EXISTS BOREAL.BRONZE.DEPOT;

CREATE ICEBERG TABLE BOREAL.BRONZE.TELEMETRIE (
    MESSAGE_ID VARCHAR, SERNR VARCHAR, MICROLOGICIEL VARCHAR, DISTANCE_KM DOUBLE,
    TEMP_MOTEUR_MAX DOUBLE, JOUR DATE)
  PARTITION BY (MONTH(JOUR))
  CATALOG = 'SNOWFLAKE'
  EXTERNAL_VOLUME = 'SNOWFLAKE_MANAGED';
```

Déposez `telemetrie_v1.parquet` dans le stage `DEPOT`, par Snowsight (menu Ingestion, Add Data, Load files into a Stage), comme à l'annexe Snowflake du Labo 03. Chargez-le, les colonnes appariées par leur nom :

```
COPY INTO BOREAL.BRONZE.TELEMETRIE
FROM @BOREAL.BRONZE.DEPOT/telemetrie_v1.parquet
FILE_FORMAT = (TYPE = PARQUET)
MATCH_BY_COLUMN_NAME = CASE_INSENSITIVE;
```

Puis faites évoluer la table comme le micrologiciel 2.0 l'a fait, déposez le second fichier, et chargez-le :

```
ALTER ICEBERG TABLE BOREAL.BRONZE.TELEMETRIE
  RENAME COLUMN TEMP_MOTEUR_MAX TO TEMPERATURE_MOTEUR_MAX;
ALTER ICEBERG TABLE BOREAL.BRONZE.TELEMETRIE ADD COLUMN HEURES_MOTEUR DOUBLE;

COPY INTO BOREAL.BRONZE.TELEMETRIE
```


© 2026 iAlexMG. Tous droits réservés.

```
FROM @BOREAL.BRONZE.DEPOT/telemetrie_v2.parquet
FILE_FORMAT = (TYPE = PARQUET)
MATCH_BY_COLUMN_NAME = CASE_INSENSITIVE;

SELECT COUNT(*), COUNT(TEMPERATURE_MOTEUR_MAX), COUNT(HEURES_MOTEUR)
FROM BOREAL.BRONZE.TELEMETRIE;
```

Si tout se passe comme au labo, la dernière requête rend 204 485, 204 485 et 115 807 : une seule colonne de température, remplie partout. Le voyage dans le temps de Snowflake s'applique à la table comme à toute autre, par `AT (OFFSET => -300)` pour l'état d'il y a cinq minutes, dans la limite de la rétention de la table, `DATA_RETENTION_TIME_IN_DAYS`, qui vaut 1 jour par défaut.

Au labo	**Sur Snowflake**
le catalogue SQL, une base SQLite	Snowflake comme catalogue, lisible des autres moteurs par Horizon
l'entrepôt Iceberg, un dossier du poste	le volume `SNOWFLAKE_MANAGED`, ou un volume externe vers votre stockage objet
`MonthTransform` sur `jour`	`PARTITION BY (MONTH(JOUR))`
`update_schema()`	`ALTER ICEBERG TABLE … RENAME COLUMN` et `ADD COLUMN`
`snapshot_from_id`	`AT (OFFSET => …)`, `AT (TIMESTAMP => …)`
compacter, expirer, nettoyer à la main	le compactage fait par Snowflake quand il écrit la table, sans frais d'après la documentation ; la rétention qui borne le voyage dans le temps

`COPY INTO` accepte aussi `LOAD_MODE = ADD_FILES_COPY`, qui copie des fichiers Parquet compatibles avec Iceberg tels quels dans la table au lieu de les réécrire, et qui exige alors `MATCH_BY_COLUMN_NAME = CASE_SENSITIVE`. Cette annexe ne montre de la plateforme que son visage Iceberg.

Annexe — Sur Databricks

Pour qui veut voir la même évolution sur la plateforme de Delta. Rien de cette annexe n'a été lancé à l'écriture du cours : les faits viennent de la documentation de Databricks, consultée le 2026-09-23. Databricks Free Edition suffit, et les commandes se tapent dans l'éditeur SQL.

Sur Databricks, une table créée sans autre précision est une table Delta gérée par Unity Catalog. `USING iceberg` en fait une table Iceberg gérée, que les autres moteurs lisent et écrivent par le catalogue REST d'Iceberg que Unity Catalog expose. Sur les tables gérées, l'optimisation prédictive lance d'elle-même `OPTIMIZE`, `VACUUM` et `ANALYZE` ; elle est active par défaut sur les comptes créés depuis le 11 novembre 2024.

© 2026 iAlexMG. Tous droits réservés.

Déposez les deux fichiers Parquet de l'annexe Snowflake dans un volume, comme à l'annexe Databricks du Labo 03 (menu New, Add or upload data, Upload files to a volume) :

```
CREATE SCHEMA IF NOT EXISTS bronze;
CREATE VOLUME IF NOT EXISTS bronze.depot;
```

Créez la table sur le premier fichier, avec la correspondance des colonnes par nom, puis faites-la évoluer et chargez le second :

```
CREATE TABLE bronze.telemetrie
TBLPROPERTIES ('delta.columnMapping.mode' = 'name')
AS SELECT message_id, sernr, micrologiciel, distance_km, temp_moteur_max, jour
   FROM read_files('/Volumes/<catalogue>/bronze/depot/telemetrie_v1.parquet');

ALTER TABLE bronze.telemetrie RENAME COLUMN temp_moteur_max TO temperature_moteur_max;
ALTER TABLE bronze.telemetrie ADD COLUMN heures_moteur DOUBLE;

INSERT INTO bronze.telemetrie
  (message_id, sernr, micrologiciel, distance_km, temperature_moteur_max, jour, heures_moteur)
SELECT message_id, sernr, micrologiciel, distance_km, temperature_moteur_max, jour, heures_moteur
FROM read_files('/Volumes/<catalogue>/bronze/depot/telemetrie_v2.parquet');
```

Remplacez `<catalogue>` par le nom du vôtre. Ce que `deltalake` 1.6 refusait à l'étape 9, la plateforme le fait : la correspondance par nom est une propriété de la table, et le renommage n'y réécrit aucun fichier. Le reste de l'étape 9 a son équivalent en SQL :

```
DESCRIBE HISTORY bronze.telemetrie;
SELECT COUNT(*) FROM bronze.telemetrie VERSION AS OF 0;
CREATE TABLE bronze.telemetrie_avant_2_0 DEEP CLONE bronze.telemetrie VERSION AS OF 0;
OPTIMIZE bronze.telemetrie;
```

Au labo	Sur Databricks
`--historique`	`DESCRIBE HISTORY`
`delta_scan(…, version => 27)`	`VERSION AS OF 27`, ou `TIMESTAMP AS OF`
`--compacter`	`OPTIMIZE`, lancé aussi par l'optimisation prédictive
`--nettoyer`, rétention forcée à 0	`VACUUM`, qui refuse sous 7 jours tant qu'on ne désactive pas le contrôle
la version 27 perdue après le nettoyage	un clone profond, copie indépendante d'une version, qui survit au `VACUUM` de la source

La documentation déconseille de tenir l'historique d'une table pour une sauvegarde : sans réglage, le voyage dans le temps n'est sûr que sur sept jours, la durée pendant laquelle `VACUUM` épargne les fichiers retirés. Enfin, une table Delta peut se faire lire en Iceberg sans copie, par UniForm, qui écrit des métadonnées

© 2026 iAlexMG. Tous droits réservés.

Iceberg à côté du journal ; les clients Iceberg la lisent alors, sans pouvoir l’écrire. UniForm exige la correspondance des colonnes, que la table ci-dessus a déjà, et des vecteurs de suppression désactivés.

© 2026 iAlexMG. Tous droits réservés.